

Tugas 13: Artificial Neural Network

Siti aisah -0110222129 ^{1*}

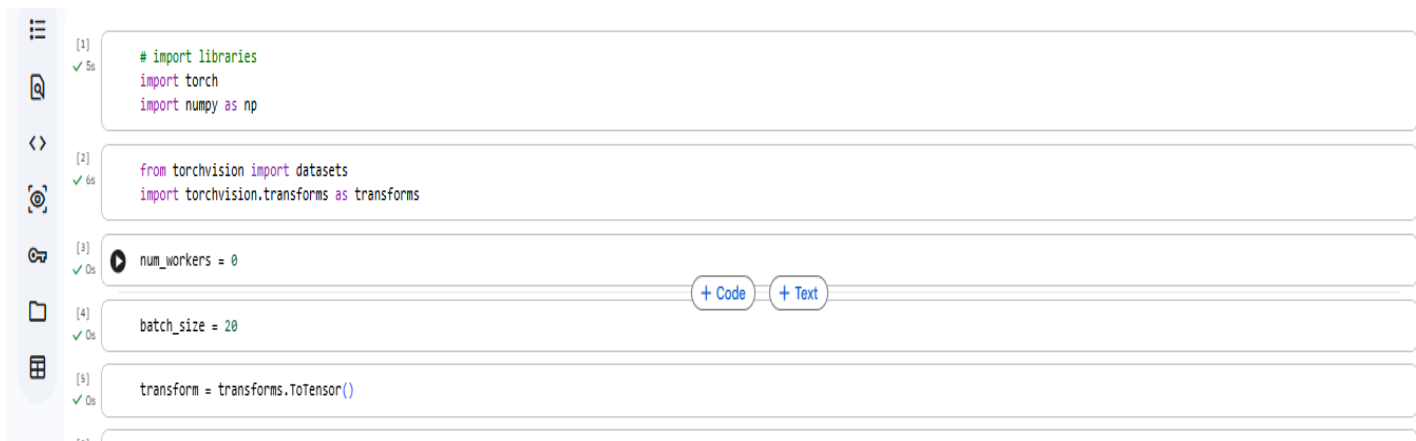
¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

Abstract.

Machine Learning (Pembelajaran Mesin) adalah cabang dari Kecerdasan Buatan (AI) yang memungkinkan sistem untuk belajar secara mandiri dari data, mengenali pola, dan membuat keputusan atau prediksi tanpa diprogram secara **eksplisit** untuk setiap

Tugas mandiri 13

1. kode untuk mempersiapkan data gambar agar dapat digunakan dalam model pembelajaran mendalam (deep learning) menggunakan PyTorch. Kode ini mengimpor pustaka yang diperlukan, mengatur parameter seperti ukuran batch dan jumlah pekerja, serta mendefinisikan transformasi untuk mengubah gambar menjadi format tensor yang sesuai untuk PyTorch.



```
[1] ✓ 5s # import libraries
import torch
import numpy as np

[2] ✓ 6s from torchvision import datasets
import torchvision.transforms as transforms

[3] ✓ 0s num_workers = 0

[4] ✓ 0s batch_size = 20

[5] ✓ 0s transform = transforms.ToTensor()
```

The screenshot shows a Jupyter Notebook interface with a sidebar on the left containing icons for file explorer, search, and other functions. The main area displays five code cells, each with a status indicator (checkmark and time) on the left. The code in the cells is as follows:

- Cell [1]: `# import libraries`, `import torch`, `import numpy as np`
- Cell [2]: `from torchvision import datasets`, `import torchvision.transforms as transforms`
- Cell [3]: `num_workers = 0`
- Cell [4]: `batch_size = 20`
- Cell [5]: `transform = transforms.ToTensor()`

Below the code cells, there are two buttons: `+ Code` and `+ Text`.

- Kode tersebut mengunduh dan memuat dataset MNIST, yang merupakan dataset standar untuk pelatihan dan pengujian model pengenalan angka tulisan tangan. Dataset ini dibagi menjadi dua bagian: data pelatihan (`train_data`) yang digunakan untuk melatih model, dan data pengujian (`test_data`) yang digunakan untuk mengevaluasi kinerja model setelah pelatihan. Transformasi yang telah didefinisikan sebelumnya diterapkan pada data saat dimuat. Proses pengunduhan dan pemuatan dataset ditunjukkan oleh bilah kemajuan (progress bar) dan kecepatan unduh.

```
train_data = datasets.MNIST(root='data', train=True,
                             download=True, transform=transform)
test_data = datasets.MNIST(root='data', train=False,
                             download=True, transform=transform)
```

100% ██████████ 9.91M/9.91M [00:00<00:00, 17.9MB/s]
100% ██████████ 28.9k/28.9k [00:00<00:00, 491kB/s]
100% ██████████ 1.65M/1.65M [00:00<00:00, 4.50MB/s]
100% ██████████ 4.54k/4.54k [00:00<00:00, 7.08MB/s]

- kode ini berfungsi untuk memuat data dalam batch dan memvisualisasikan beberapa contoh dari data pelatihan, yang berguna untuk memeriksa data dan memastikan bahwa data dimuat dengan benar sebelum memulai pelatihan model.

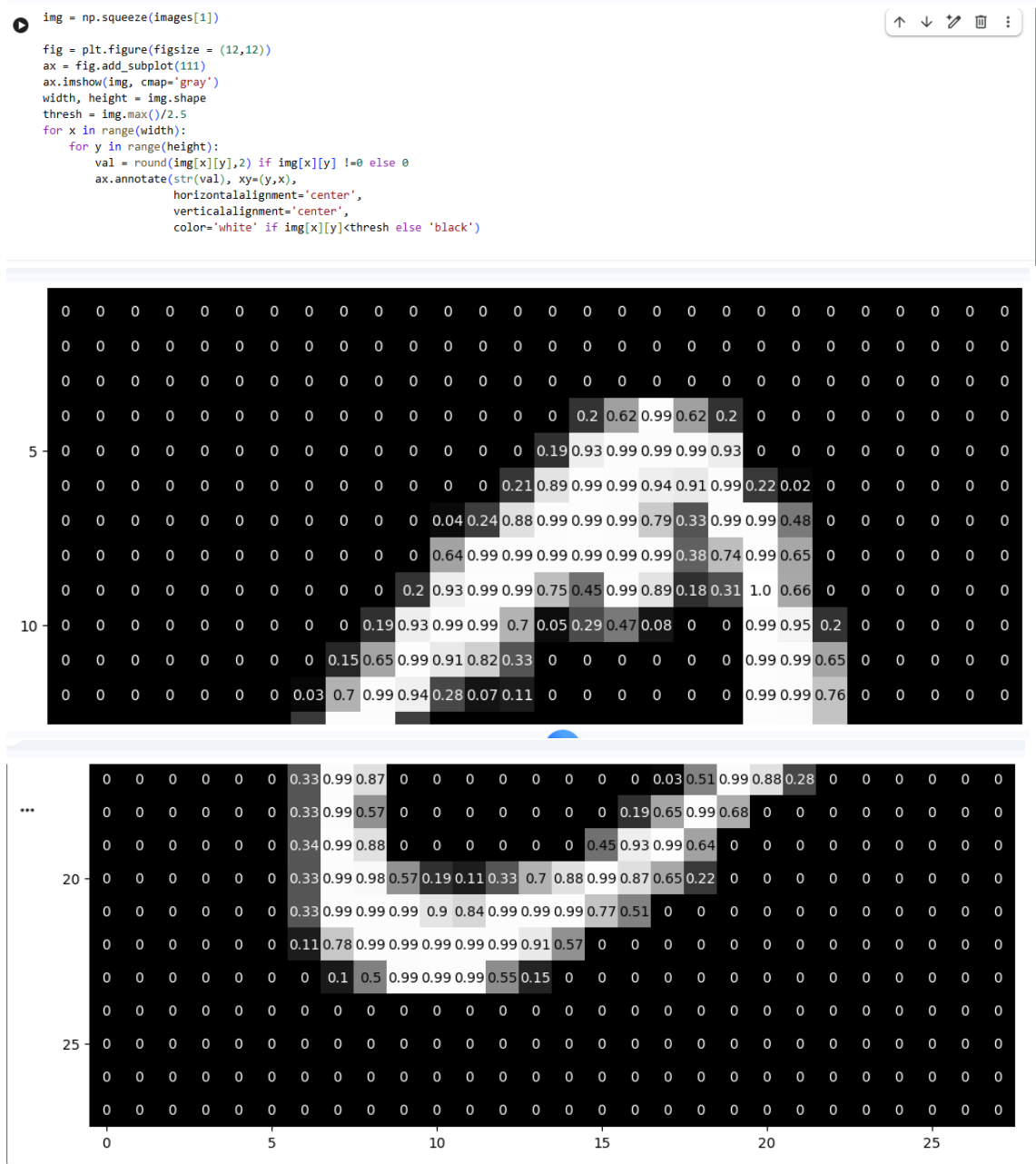
```
[7] ✓ On train_loader = torch.utils.data.DataLoader(train_data, batch_size=batch_size,
num_workers=num_workers)
test_loader = torch.utils.data.DataLoader(test_data, batch_size=batch_size,
num_workers=num_workers)

[12] ✓ On import matplotlib.pyplot as plt
%matplotlib inline

# obtain one batch of training images
dataiter = iter(train_loader)
images, labels = next(dataiter)
images = images.numpy()

# plot the images in the batch, along with the corresponding labels
fig = plt.figure(figsize=(25, 4))
for idx in np.arange(20):
    ax = fig.add_subplot(2, 20//2, idx+1, xticks=[], yticks=[])
    ax.imshow(np.squeeze(images[idx]), cmap='gray')
    # print out the correct label for each image
    # .item() gets the value contained in a Tensor
    ax.set_title(str(labels[idx].item()))
```

- kode ini mengambil gambar, menampilkan gambar tersebut menggunakan Matplotlib, dan kemudian menambahkan anotasi ke setiap piksel gambar yang menunjukkan nilai piksel tersebut. Warna anotasi (putih atau hitam) ditentukan berdasarkan apakah nilai piksel kurang dari ambang batas yang dihitung dari nilai maksimum piksel dalam gambar. Kode ini kemungkinan digunakan untuk memvisualisasikan nilai piksel dalam gambar dan memberikan representasi visual dari intensitas piksel.



5. mendefinisikan jaringan saraf dengan tiga lapisan linier, fungsi aktivasi ReLU, dan lapisan dropout. Jaringan saraf ini dirancang untuk klasifikasi, khususnya untuk dataset MNIST. Metode forward mendefinisikan bagaimana input diteruskan melalui lapisan-lapisan ini untuk menghasilkan prediksi.

```
[15]
✓ Os
import torch.nn as nn
import torch.nn.functional as F

## TODO: Define the NN architecture
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        # linear layer (784 -> 1 hidden node)
        self.fc1 = nn.Linear(28 * 28, 512)
        self.fc2 = nn.Linear(512, 512)
        self.fc3 = nn.Linear(512, 10)
        self.dropout = nn.Dropout(p=0.2)

    def forward(self, x):
        # flatten image input
        x = x.view(-1, 28 * 28)
        # add hidden layer, with relu activation function
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.dropout(F.relu(self.fc2(x)))
        x = self.fc3(x)
        return x

[15]
✓ Os
# initialize the NN
model = Net()
print(model)

... Net(
  (fc1): Linear(in_features=784, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=512, bias=True)
  (fc3): Linear(in_features=512, out_features=10, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
```

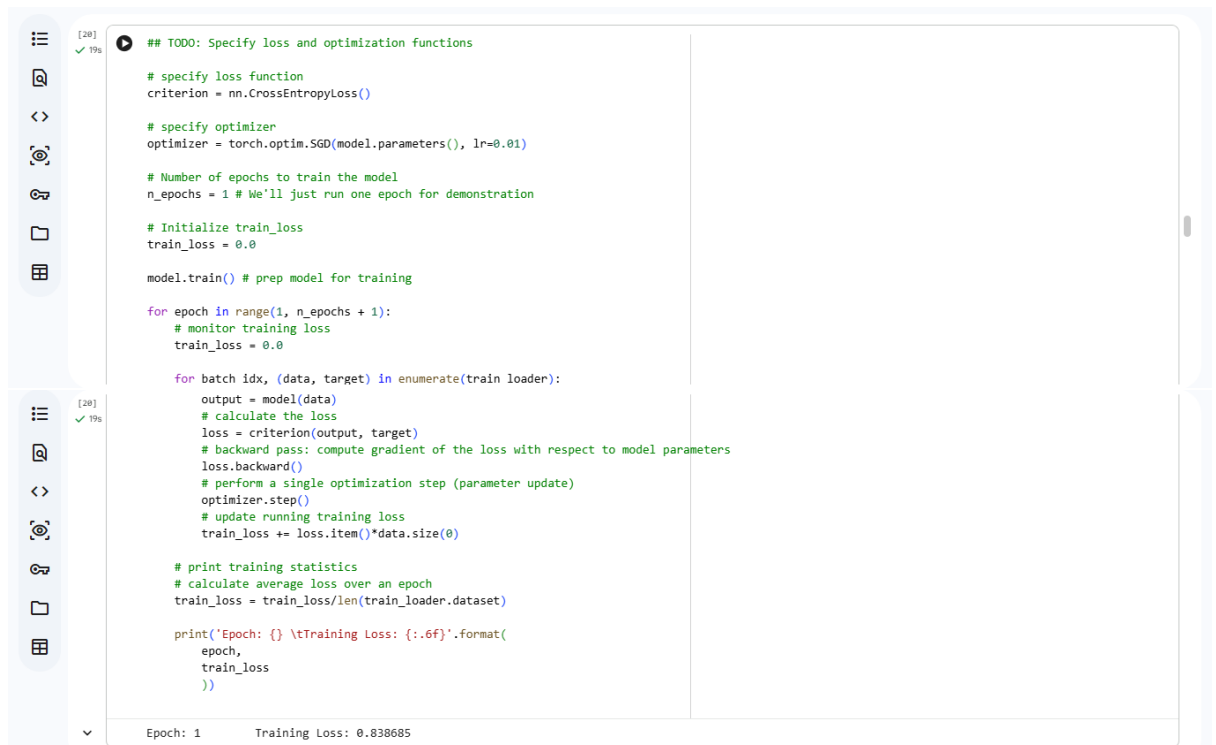
6. kode ini adalah untuk mempersiapkan proses pelatihan (training) sebuah model jaringan saraf (neural network). Secara spesifik,

```
[16]
✓ Os
## TODO: Specify loss and optimization functions

# specify loss function
criterion = nn.CrossEntropyLoss()

# specify optimizer
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

7. kode ini melakukan iterasi melalui dataset pelatihan, memasukkan data ke dalam model, menghitung loss, menghitung gradient, dan memperbarui bobot model untuk mengurangi loss. Proses ini diulangi untuk setiap batch data dan setiap epoch, sehingga model secara bertahap belajar untuk membuat prediksi yang lebih akurat.



```
## TODO: Specify loss and optimization functions

# specify loss function
criterion = nn.CrossEntropyLoss()

# specify optimizer
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# Number of epochs to train the model
n_epochs = 1 # We'll just run one epoch for demonstration

# Initialize train_loss
train_loss = 0.0

model.train() # prep model for training

for epoch in range(1, n_epochs + 1):
    # monitor training loss
    train_loss = 0.0

    for batch_idx, (data, target) in enumerate(train_loader):
        output = model(data)
        # calculate the loss
        loss = criterion(output, target)
        # backward pass: compute gradient of the loss with respect to model parameters
        loss.backward()
        # perform a single optimization step (parameter update)
        optimizer.step()
        # update running training loss
        train_loss += loss.item()*data.size(0)

    # print training statistics
    # calculate average loss over an epoch
    train_loss = train_loss/len(train_loader.dataset)

    print('Epoch: {} \tTraining Loss: {:.6f}'.format(
        epoch,
        train_loss
    ))
```

Epoch: 1 Training Loss: 0.838685

8. kode ini memasukkan data pengujian ke dalam model, membandingkan prediksi model dengan label sebenarnya, dan menghitung berbagai metrik kinerja, seperti loss pengujian, akurasi per kelas, dan akurasi keseluruhan. Ini memberikan gambaran tentang seberapa baik model melakukan generalisasi ke data yang tidak pernah dilihat sebelumnya

```
[21] ✓ 1s
test_loss = 0.0
class_correct = list(0. for i in range(10))
class_total = list(0. for i in range(10))

model.eval() # prep model for *evaluation*

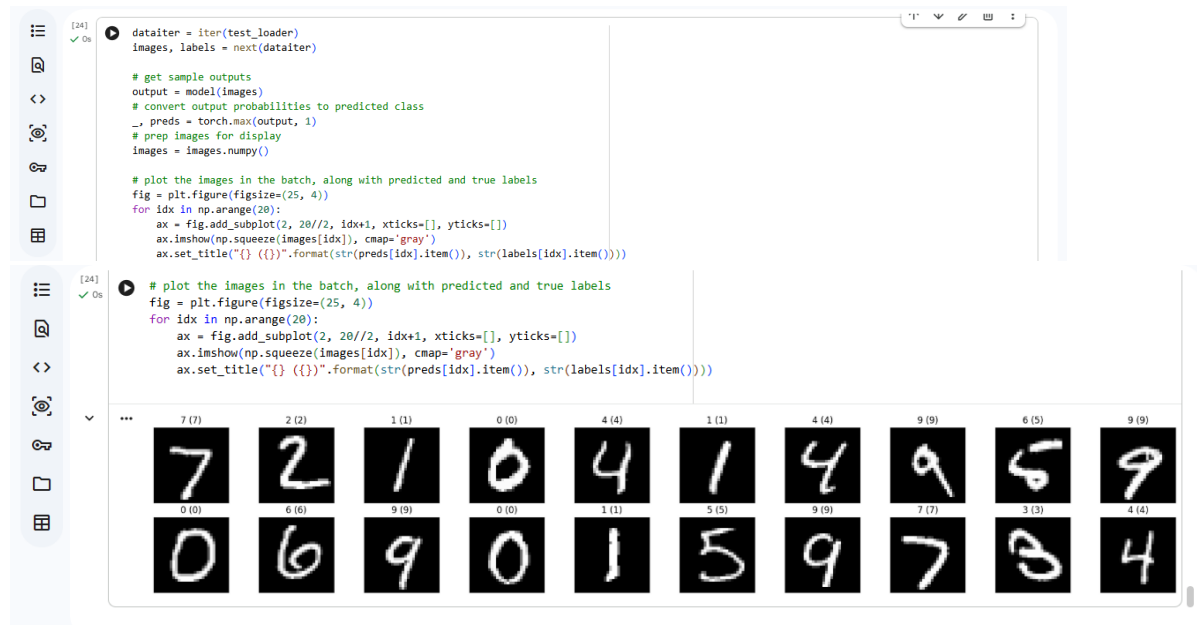
for data, target in test_loader:
    # forward pass: compute predicted outputs by passing inputs to the model
    output = model(data)
    # calculate the loss
    loss = criterion(output, target)
    # update test loss
    test_loss += loss.item()*data.size(0)
    # convert output probabilities to predicted class
    _, pred = torch.max(output, 1)
    # compare predictions to true label
    correct = np.squeeze(pred.eq(target.data.view_as(pred)))
    # calculate test accuracy for each object class
    for i in range(batch_size):
        label = target.data[i]
        class_correct[label] += correct[i].item()
        class_total[label] += 1

if class_total[i] > 0:
    print('Test Accuracy of %5s: %2d%% (%2d/%2d)' % (
        str(i), 100 * class_correct[i] / class_total[i],
        np.sum(class_correct[i]), np.sum(class_total[i])))
else:
    print('Test Accuracy of %5s: N/A (no training examples)' % (classes[i]))
print('\nTest Accuracy (Overall): %2d%% (%2d/%2d)' % (
    100. * np.sum(class_correct) / np.sum(class_total),
    np.sum(class_correct), np.sum(class_total)))

Test Loss: 0.343007

Test Accuracy of 0: 98% (962/980)
Test Accuracy of 1: 98% (1119/1135)
Test Accuracy of 2: 83% (865/1032)
Test Accuracy of 3: 88% (891/1010)
Test Accuracy of 4: 92% (988/982)
Test Accuracy of 5: 85% (762/892)
Test Accuracy of 6: 91% (881/958)
Test Accuracy of 7: 85% (878/1028)
Test Accuracy of 8: 82% (807/974)
Test Accuracy of 9: 91% (920/1009)
```

9. kode ini mengambil beberapa sampel data pengujian, memasukkan gambar-gambar tersebut ke dalam model untuk mendapatkan prediksi, dan kemudian menampilkan gambar-gambar tersebut bersama dengan prediksi model dan label sebenarnya. Ini memungkinkan kita untuk melihat secara visual bagaimana model melakukan prediksi dan membandingkannya dengan label yang benar.



Referensi:

- Munir, S., Seminar, K. B., Sudradjat, Sukoco, H., & Buono, A. (2022). The Use of Random Forest Regression for Estimating Leaf Nitrogen Content of Oil Palm Based on Sentinel 1-A Imagery. *Information*, 14(1), 10. <https://doi.org/10.3390/info14010010>
- Seminar, K. B., Imantho, H., Sudradjat, Yahya, S., Munir, S., Kaliana, I., Mei Haryadi, F., Noor Baroroh, A., Supriyanto, Handoyo, G. C., Kurnia Wijayanto, A., Ijang Wahyudin, C., Liyantono, Budiman, R., Bakir Pasaman, A., Rusiawan, D., & Sulastri. (2024). PreciPalm: An Intelligent System for Calculating Macronutrient Status and Fertilizer Recommendations for Oil Palm on Mineral Soils Based on a Precision Agriculture Approach. *Scientific World Journal*, 2024(1). <https://doi.org/10.1155/2024/1788726>

LINK GITHUB: <https://github.com/Sitiaisah1604/machine-learning>